

CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

GIẢI THUẬT SẮP XẾP NỖI BỌT – CHÈN – CHỌN



TS. TRẦN NGỌC VIỆT
TH.S LÊ CÔNG HIẾU



HỌC KỲ 3 – NĂM HỌC 2025-2026



KHÓA K30 & K30, K31



NỘI DUNG

- *Giới thiệu bài toán sắp xếp**
- *Sắp xếp nổi bọt – Bubble Sort**
- *Sắp xếp chèn – Insertion Sort**
- *Sắp xếp chọn – Selection Sort**



GIỚI THIỆU

- Sắp xếp là gì? Trong thực tế chúng ta đã từng thấy những thứ gì được sắp xếp? Giá trị của việc sắp xếp mang lại là gì?
 - Danh sách học sinh trong lớp
 - Danh sách xếp hạng điểm người chơi
 - Các sản phẩm trên thương mại điện tử
 - Tra cứu từ điển...
- Trong phần này các thuật toán sắp xếp sẽ dùng các số nguyên để biểu diễn.



BÀI TOÁN SẮP XẾP

- Cho danh sách có n phần tử $a_0, a_1, a_2, \dots, a_{n-1}$.
- Sắp xếp là quá trình xử lý các phần tử trong danh sách để đặt chúng theo một thứ tự thỏa mãn một số tiêu chuẩn nào đó dựa trên thông tin lưu tại mỗi phần tử, như:
 - Sắp xếp danh sách lớp học tăng theo điểm trung bình.
 - Sắp xếp danh sách sinh viên tăng theo tên.
- Để đơn giản trong việc trình bày giải thuật ta dùng mảng 1 chiều a để lưu danh sách trên trong bộ nhớ chính.

BÀI TOÁN SẮP XẾP (tt)

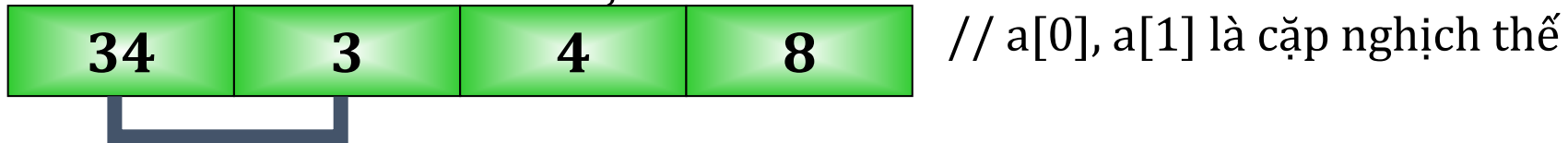
a: là dãy các phần tử dữ liệu

Để sắp xếp dãy a theo thứ tự (theo thứ tự tăng dần), ta tiến hành triệt tiêu tất cả các nghịch thế trong a.

▪ Nghịch thế:

- Cho dãy có n phần tử $a_0, a_1, \dots, a_{n-1}$

- Nếu $i < j$ và $a_i > a_j$



Đánh giá độ phức tạp của giải thuật, ta tính

C_{ss} : Số lượng phép so sánh cần thực hiện

C_{HV} : Số lượng phép hoán vị cần thực hiện



CÁC THUẬT TOÁN SẮP XẾP ĐƠN GIẢN

- Các thuật toán sắp xếp được trình bày ở đây gồm:
 - Thuật toán sắp xếp kiểu nổi bọt (Bubble Sort)
 - Thuật toán sắp xếp kiểu lựa chọn (Selection Sort)
 - Thuật toán sắp xếp kiểu chèn (Insertion Sort)



01. BUBBLE SORT

- Xuất phát từ đầu dãy, đổi chỗ các cặp phần tử kế cận để đưa phần tử nhỏ hơn trong cặp phần tử đó về vị trí đứng đầu dãy hiện hành, sau đó sẽ không xét đến nó ở bước tiếp theo, do vậy ở lần xử lý thứ i sẽ có vị trí đầu dãy là i .
- Lặp lại xử lý trên cho đến khi không còn cặp phần tử nào để xét.



CÁC BƯỚC THUẬT TOÁN (tham khảo)

Bước 1 : $i = 0$; //đầu tiên

Bước 2 : $j = i$; //duyệt từ đầu dãy đến cuối dãy vị trí $n-1$;
danh sách có n phần tử $a_0, a_1, a_2 \dots, a_{n-1}$.

Trong khi ($j > i$) thực hiện:

 Nếu $a[j] < a[j-1]$

 Doicho($a[j], a[j-1]$);

$j = j-1$;

Bước 3 : $i = i+1$; // lần xử lý kế tiếp

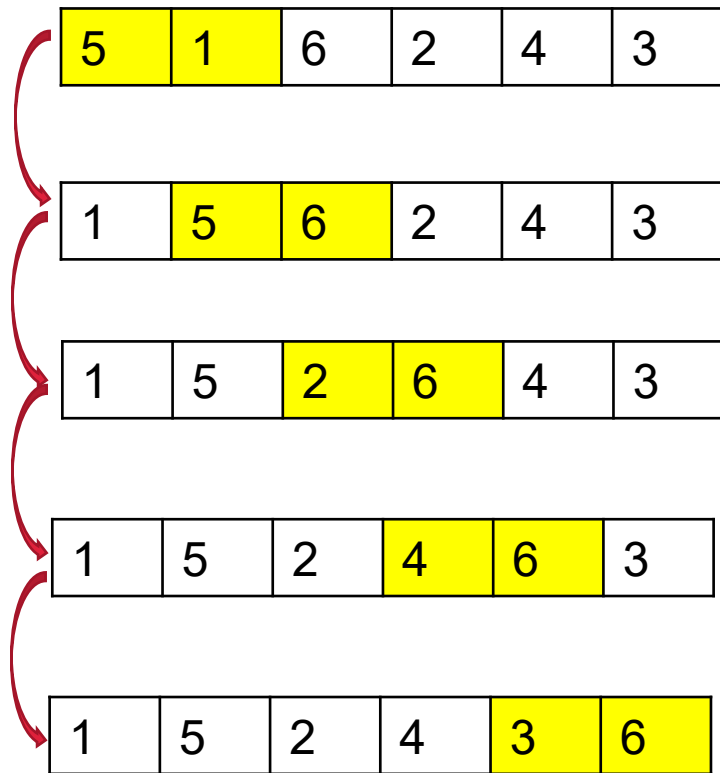
 Nếu $i \geq n-1$: Hết dãy. Dừng kết thúc.

 Ngược lại, quay lui Bước 2.

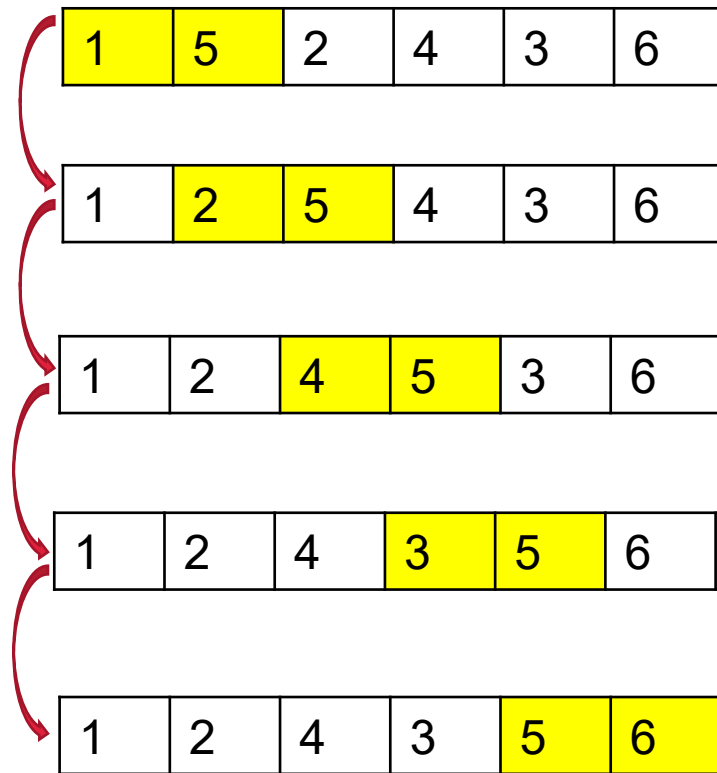
MINH HỌA THUẬT TOÁN

- Cho 1 dãy các phần tử như sau: {5, 1, 6, 2, 4, 3}

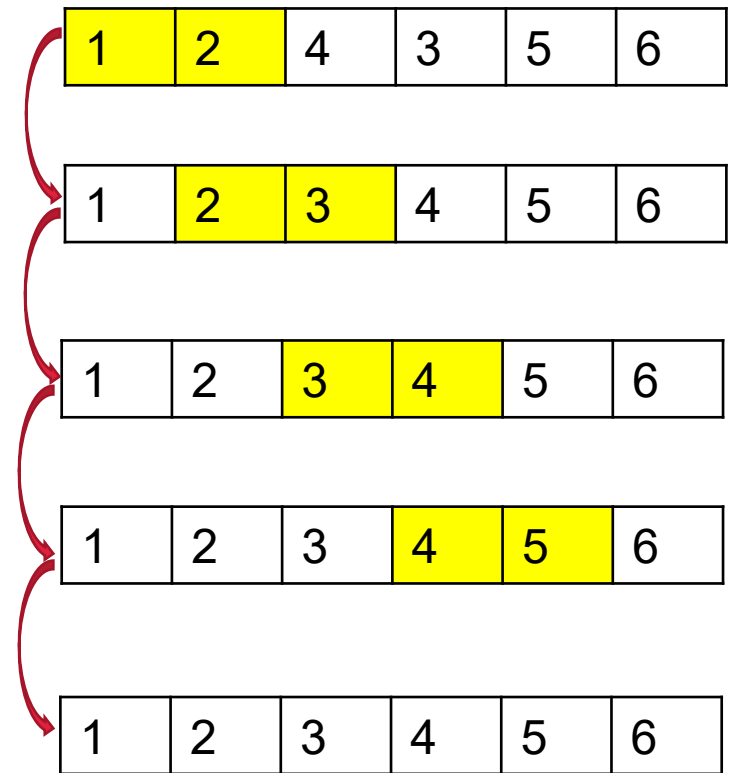
Lượt 1



Lượt 2



Lượt 3





01. BUBBLE SORT

- Xuất phát từ đầu dãy, đổi chỗ các cặp phần tử kế cận để đưa phần tử nhỏ hơn trong cặp phần tử đó về vị trí đứng đầu dãy hiện hành, sau đó sẽ không xét đến nó ở bước tiếp theo, do vậy ở lần xử lý thứ i sẽ có vị trí đầu dãy là i .
- Lặp lại xử lý trên cho đến khi không còn cặp phần tử nào để xét.



BUBBLE SORT ALGORITHM:

#Giải thuật Nổi bọt - Bubble Sort:

B.1: Gán $i = 0$

B.2: Gán $j = 0$ //danh sách có n phần tử $a_0, a_1, a_2, \dots, a_{n-1}$

B.3: Nếu $A[j] > A[j + 1]$ thì Hoán đổi chỗ giữa $A[j]$ và $A[j + 1]$

B.4: Nếu $(j < N - i - 1)$: #N= n-1

- Đúng thì $j = j + 1$ và quay lui bước 3

- Sai thì chuyển sang bước 5

B.5: Nếu $(i < N - 1)$:

- Đúng thì $i = i + 1$ và quay lui bước 2

- Sai thì dừng Kết Thúc.

Ví dụ 01: Áp dụng giải thuật bubble_sort. Thực hiện sắp xếp mảng sau

Cho Mảng A = [19,13,6]

+B.1: $i = 0$;

B.2: $j = 0$;

B.3: nếu $A[j] > A[j+1]$ ($A[0] > A[1]$; $19 > 13$) thì Hoán đổi giữa 19 và 13
[13,19,6];

B.4: nếu $j < (n-i-1)$ ($0 < (2-0-1)$): Đúng thì $j = j+1 = 0+1 = 1$

#N = n-1 = 2

và quay lui B.3;

+B.3: nếu $A[j] > A[j+1]$ ($A[1] > A[2]$; $19 > 6$) thì Hoán đổi giữa 19 và 6
[13, 6, 19];

B.4: nếu $j < (n-i-1)$ ($1 < (2-0-1)$): Sai thì chuyển sang B.5

B.5: nếu $i < (n-1)$ ($0 < 1$): Đúng thì $i = i+1 = 0 + 1 = 1$ và quay lại B.2

+B.2: $j = 0$ ($i = 1$);

B.3: nếu $A[0] > A[1]$ ($13 > 6$) thì Hoán đổi giữa 13 và 6
[6, 13, 19];

B.4: Nếu $(j < n - i - 1)$ ($0 < 0$): Sai thì chuyển sang B.5

B.5: Nếu $(i < n - 1)$ ($1 < 1$): Sai thì Dừng Kết thúc.

Vậy mảng được sắp xếp là:

[6, 13, 19].

Ví dụ 02: Áp dụng giải thuật bubble_sort. Thực hiện sắp xếp mảng sau

A = [40, 15, 30]

Giải

+i=0; j=0

Nếu $A[0] > A[1]$ ($40 > 15$) thì hoán đổi 40 và 15. Mảng **A = [15, 40, 30]**

Nếu $(j < n-i-1)$ ($0 < 2-0-1$): $j = j + 1 = 0 + 1 = 1$ và quay về B.3

#N = n-1=2

+i=0; j=1

Nếu $A[j] > A[j+1]$ ($A[1] > A[2]$) $40 > 30$ thì hoán đổi 40 và 30. Mảng

A = [15, 30, 40]

Nếu $(j < n-i-1)$ ($1 < 2-0-1$) Sai

Nếu $(i < n-1)$ ($0 < 1$) Đúng. $i = i + 1$ và quay về B.2

+j=0; i=1

Nếu $A[0] > A[1]$ Sai, nên giữ nguyên mảng **A = [15, 30, 40]**

Nếu $(j < n-i-1)$ ($0 < 2-1-1$) Sai. Kết thúc, vậy mảng được sắp xếp là

A = [15, 30, 40]



BUBBLE SORT ALGORITHM:

BT 01: Áp dụng giải thuật Bubble Sort – Nổi bọt. Thực hiện sắp xếp mảng

A = [72, 45, 36]

BT 02: Áp dụng giải thuật Bubble Sort – Nổi bọt. Thực hiện sắp xếp mảng

A = [102, 65, 76, 15]



* Đánh giá độ phức tạp - thuật toán Bubble Sort

```
void BubbleSort()  
{  
    int i, j ;  
    for (i = 0; i < n; i++)  
        for ( j = n-1; j > i; j--)  
            if (a[j] > a[j-1]) //hoán chuyển  
                swap(&a[j], &a[j-1]);  
}
```

#Độ phức tạp của thuật toán Bubble Sort là $T(n) = O(n^2)$. Trường hợp xấu nhất xảy ra là khi mảng được sắp xếp ngược lại.



02. INSERTION SORT

- Giả sử có một dãy $a_0, a_1, \dots, a_{n-1}$ trong đó i phần tử đầu tiên $a_0, a_1, \dots, a_{i-1}$ đã có thứ tự (dãy con).
- Tìm cách chèn phần tử a_i vào **vị trí thích hợp** của đoạn đã được sắp để có dãy mới $a_0, a_1, \dots, a_i$ trở nên có thứ tự. Vị trí này chính là vị trí giữa hai phần tử a_{k-1} và a_k thỏa mãn $a_{k-1} < a_i < a_k$ ($1 \leq k \leq i$).



02. INSERTION SORT

- Sắp xếp chèn là một giải thuật sắp xếp dựa trên so sánh tại chỗ.
- Một danh sách con luôn luôn được duy trì dưới dạng đã qua sắp xếp. Sắp xếp chèn là chèn thêm một phần tử vào danh sách con đã qua sắp xếp.
- Phần tử được chèn vào vị trí thích hợp sao cho vẫn đảm bảo rằng danh sách con đó vẫn sắp xếp theo thứ tự.
- Giải thuật này không thích hợp sử dụng với các tập dữ liệu lớn khi độ phức tạp trường hợp xấu nhất và trường hợp trung bình là $O(n^2)$ với n là số phần tử.

02. INSERTION SORT

Ví dụ: cho một mảng gồm các phần tử không có thứ tự:

14 - 33 - 27 - 10 - 35 - 19 - 42 - 44

-Giải thuật sắp xếp chèn so sánh hai phần tử đầu tiên:

14 - 33 - 27 - 10 - 35 - 19 - 42 - 44

-Giải thuật tìm ra rằng cả 14 và 33 đều đã trong thứ tự tăng dần. Bây giờ, 14 là trong danh sách con đã qua sắp xếp.

14 - 33 - 27 - 10 - 35 - 19 - 42 - 44

-Giải thuật sắp xếp chèn tiếp tục di chuyển tới phần tử kế tiếp và so sánh 33 và 27.

14 - **33 - 27** - 10 - 35 - 19 - 42 - 44

-Và thấy rằng 33 không nằm ở vị trí đúng.

14 - **33** - 27 - 10 - 35 - 19 - 42 - 44

-Giải thuật sắp xếp chèn trao đổi vị trí của 33 và 27. Kiểm tra tất cả phần tử trong danh sách con đã sắp xếp. Thấy rằng trong danh sách con này chỉ có một phần tử 14 và 27 là lớn hơn 14. Do vậy danh sách con vẫn giữ nguyên sau khi đã trao đổi.

14 - 27 - 33 - 10 - 35 - 19 - 42 - 44

-Danh sách con chúng ta có hai giá trị 14 và 27. Tiếp tục so sánh 33 với 10.

14 - 27 - **33 - 10** - 35 - 19 - 42 - 44

02. INSERTION SORT

-Hai giá trị này không theo thứ tự.

14 - 27 - 33 - 10 - 35 - 19 - 42 - 44

-Vì thế chúng ta trao đổi chúng.

14 - 27 - 10 - 33 - 35 - 19 - 42 - 44

-Việc trao đổi dẫn đến 27 và 10 không theo thứ tự.

14 - 27 - 10 - 33 - 35 - 19 - 42 - 44

-Vì thế chúng ta cũng trao đổi chúng.

14 - 10 - 27 - 33 - 35 - 19 - 42 - 44

-Chúng ta lại thấy rằng 14 và 10 không theo thứ tự.

14 - 10 - 27 - 33 - 35 - 19 - 42 - 44

-Và chúng ta tiếp tục trao đổi hai số này. Cuối cùng, sau vòng lặp thứ 3 chúng ta có 4 phần tử.

10 - 14 - 27 - 33 - 35 - 19 - 42 - 44

-Tiến trình trên sẽ tiếp tục diễn ra cho tới khi tất cả giá trị chưa được sắp xếp được sắp xếp hết vào trong danh sách con đã qua sắp xếp. Tiếp theo chúng ta cùng tìm hiểu khía cạnh lập trình của giải thuật sắp xếp chèn.

10 - 14 - 19 - 27 - 33 - 35 - 42 - 44



02. INSERTION SORT

Giải thuật sắp xếp chèn - Insertion Sort

Bước 1: Kiểm tra nếu phần tử đầu tiên đã được sắp xếp.
trả về 1

Bước 2: Lấy phần tử kế tiếp

Bước 3: So sánh với tất cả phần tử trong danh sách con đã qua sắp xếp

Bước 4: Dịch chuyển tất cả phần tử trong danh sách con mà lớn hơn giá trị để được sắp xếp

Bước 5: Chèn giá trị đó

Bước 6: Lặp lại cho tới khi danh sách được sắp xếp



02. INSERTION SORT (tham khảo)

#Thuật toán insertion sort

Để sắp xếp một mảng có kích thước n theo thứ tự tăng dần:

B.1. Lặp lại từ $arr[1]$ đến $arr[n]$ trên mảng.

B.2. So sánh phần tử hiện tại (key) với phần tử tiền nhiệm của nó.

B.3. Nếu phần tử chính nhỏ hơn phần tử trước của nó, hãy so sánh nó với các phần tử trước đó. Di chuyển các phần tử lớn hơn lên một vị trí để tạo khoảng trống cho phần tử được hoán đổi.



03. SELECTION SORT

#Thuật toán Selection Sort

Thuật toán Selection Sort sắp xếp một mảng bằng cách liên tục tìm phần tử nhỏ nhất (xét theo thứ tự tăng dần), từ phần chưa được sắp xếp và đặt nó ở đầu. Thuật toán duy trì hai mảng con trong một mảng nhất định.

1. Mảng con đã được sắp xếp.

2. Mảng con còn lại chưa được sắp xếp.

Trong mỗi lần lặp lại sắp xếp lựa chọn, phần tử tối thiểu (xét theo thứ tự tăng dần) từ mảng con chưa được sắp xếp sẽ được chọn và chuyển đến mảng con đã sắp xếp.

03. SELECTION SORT

#Ví dụ: cho một mảng gồm các phần tử không có thứ tự

```
//Dãy chưa sắp xếp  
arr[] = 84 45 12 32 10
```

```
// Tìm phần tử bé nhất trong dãy arr[0...4]  
// đưa phần tử đó lên đầu dãy arr[0...4]  
**10** 45 12 32 84
```

```
// Tìm phần tử bé nhất trong dãy arr[1...4]  
// đưa phần tử đó lên đầu dãy arr[1...4]  
10 **12** 45 32 84
```

```
// Tìm phần tử bé nhất trong dãy arr[2...4]  
// đưa phần tử đó lên đầu dãy arr[2...4]  
10 12 **32** 45 84
```

```
// Tìm phần tử bé nhất trong dãy arr[3...4]  
// đưa phần tử đó lên đầu dãy arr[3...4]  
10 12 32 **45** 84
```

BÀI KIỂM TRA SỐ 3

Câu 1:

cho một mảng gồm các phần tử không có thứ tự:

14 – 33 – 27 – 110 – 135 – 19

a) Biểu diễn Giải thuật sắp xếp chèn

b) Biểu diễn giải thuật sắp xếp Nổi bọt



BÀI KIỂM TRA SỐ 04

BT 01:

a) Áp dụng giải thuật sắp xếp nổi bọt (Bubble Sort).

Thực hiện sắp xếp mảng **arr** = [400, 250, 30, 55]

b) Áp dụng giải thuật sắp xếp chọn (Selection Sort).

Thực hiện sắp xếp mảng **arr** = [260, 95, 40, 49, 35, 45, 36, 18]



#Thực hành 01.1:

#Bubble Sort

```
>>> def bubble_sort(arr):           #??
    for i in range(len(arr)):        #??
        for j in range(len(arr) - 1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                #??
```

```
>>> arr = [120, 35, 60, 42, 280, 7, 15, 19]
```

```
>>> bubble_sort(arr)
```

```
>>> print(arr)
```

#Nhan xet: input & output cua Giai thuat tren ?

#Thực hành 01.2:

```
#bubble_sort;
def bubble_Sort(arr):
    n = len(arr)    //độ dài của arr
    swapped = False
    for i in range(n-1):
        for j in range(0, n-i-1):
            if arr[j] > arr[j + 1]:
                swapped = True
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                //hoán đổi
        if not swapped:
            return
arr = [60, 32, 15, 12, 52, 71, 90, -1, -10, -30, -155, 75]
bubble_Sort(arr)
print("mang duoc sap xep la:")
for i in range(len(arr)):
    print("% d" % arr[i], end=" ")
```

#nhận xét: input, output của giải thuật

#Thực hành 01.3:

```
#bubble_sort;
def bubbleSort(arr):
    for i in range(len(arr)):
        for j in range(0, len(arr) - i - 1):
            if arr[j] > arr[j + 1]:
                temp = arr[j]
                arr[j] = arr[j+1]
                arr[j+1] = temp
arr = [25, 17, 7, 14, 6, 3, 100, -2,-10,-50]
print("mang chua duoc sap xep la: ", arr )|
bubbleSort(arr)
print('mang duoc sap xep la: ', arr)

mang chua duoc sap xep la: [25, 17, 7, 14, 6, 3, 100, -2, -10, -50]
mang duoc sap xep la: [-50, -10, -2, 3, 6, 7, 14, 17, 25, 100]
```

#nhận xét: input, output của giải thuật



#Thực hành 01.4:

#thuật toán bubble sort – kết quả đầu ra là
mảng sắp xếp từ lớn đến bé

.....

#Thực hành 02.1:

```
#insertion sort
def insertion_s(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        # Di chuyển các phần tử của arr[0..i-1], lớn hơn key,
        # đến một vị trí phía trước vị trí hiện tại
        j = i-1
        while (j >= 0 and key < arr[j]):
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

arr = [12, 31, 130, 15, 18, 150, 30]
insertion_s(arr)
for i in range(len(arr)):
    print ("% d" % arr[i])
```

#nhận xét: input, output của giải thuật



#Thực hành 02.2:

```
def insertion_sort(arr):
    n = len(arr)

    if n <= 1:
        return #Nếu mảng có 0 hoặc 1 phần tử, thì mảng đã được sắp xếp

    for i in range(1, n):
        key = arr[i] #Lưu trữ phần tử hiện tại làm khóa để chèn vào đúng vị trí
        j = i-1
        while (j >= 0 and key < arr[j]):
            #Di chuyển các phần tử lớn hơn khóa một vị trí về phía trước;
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key

arr = [12, 11, 13, 58, 6, 90, 55]
insertion_sort(arr)
print(arr)
```

#nhận xét: input, output của giải thuật



#Thực hành 03.1:

```
#selection sort
```

```
import sys
```

```
arr = [140, 25, 15, 52, 10, 250, 55]
```

```
for i in range(len(arr)):
```

```
    #Tìm kiếm phần tử bé nhất trong dãy
```

```
    min = i
```

```
    for j in range(i+1, len(arr)):
```

```
        if arr[min] > arr[j]:
```

```
            min = j
```

```
    #Đổi chỗ phần tử
```

```
    arr[i], arr[min] = arr[min], arr[i]
```

```
print ("Sorted array")
```

```
for i in range(len(arr)):
```

```
    print("%d" %arr[i])
```

#nhận xét: input, output của giải thuật



#Thực hành 03.2:

```
def selection_Sort(array, n):  
  
    for index in range(n):  
        min_index = index  
  
        for j in range(index + 1, n):  
            #chọn phần tử nhỏ nhất mỗi lần lặp  
            if array[j] < array[min_index]:  
                min_index = j  
            #hoán đổi các phần tử để sắp xếp mảng  
            (array[index], array[min_index]) = (array[min_index], array[index])  
  
arr = [-25, 40, 0, 15, -90, 80, -92, -210, 747, 500, 1050, 999]  
n = len(arr)  
selection_Sort(arr, n)  
print('array sau khi sắp xếp là:')  
print(arr)
```

#nhận xét: input, output của giải thuật



#Thực hành 03.3:

```
arr = [64, 34, 25, 5, 22, 11, 90, 12, -15, -50, 100]

n = len(arr)
for i in range(n-1):
    min_index = i
    for j in range(i+1, n):
        if arr[j] < arr[min_index]:
            min_index = j
    min_value = arr.pop(min_index)
    arr.insert(i, min_value)

print("Sorted array:", arr)
```

Sorted array: [-50, -15, 5, 11, 12, 22, 25, 34, 64, 90, 100]

#nhận xét: input, output của giải thuật



#Thực hành 03.4:

#thuật toán sắp xếp chọn – mảng kiểu string

.....



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

